

Jake Takahashi, Isaac Jee, Jesus Hernandez Chairez, Aaron Castiglione
Dr. Careira
CSE 176
17 December 2025

Part 1 Report

After examining our data .m file with a quick python script, we saw that the data contained four arrays, called `train_fea1`, `train_gnd1`, `test_fea1`, and `test_gn`. However, we first need to clean and preprocess our data.

We then used this information, and created a script to examine the shape of the data using numpy's built in `.shape()` function, which we appropriately named `feature.py`. We can see that the `train_fea1` data is organized in a 2D array of shape (6000, 100); this is a 2D array. This is fine and we can pass it into our model as is. However, our `train_gnd1` data is shown to have a 2D shape as well; more specifically, of shape (6000,1). This is a natural result of how arrays are stored in matlab, but most online python machine learning libraries require ground truth label arrays to be in 1D, so we use `.flatten` to essentially force our class label array to be 1D.

The next step of our data processing is to filter out only the data we need. The project instructions say to use images 1-1000 as training images, images 1001-2000 as validation images, and 2001-3000 as test images. However, we first need to filter out all of the images with a label of 5 and 6 in the first place. To do that, we defined something called a mask on our labels array. Our mask's defined "condition" is if the label at the label array's index is 5 or 6, then true, and false otherwise. We then applied that mask to our `train_fea1` array to fully filter out the images with only a label of 5 or 6, and I also filtered out my class label array, while also defining 5 to be the negative class while 6 is the positive class.

The next step was to filter out images 1-1000, 1001-2000, and 2001-3000 as our training, validation, and test sets, respectively. Python has a built-in function to do this. However, there is one problem with this. In the `MNISTmini.mat` file, data with the same class labels are grouped together. All the images with label 5 are grouped together, same with 6, etc. So, if we were to simply take images 1-1000, 1001-200, and 2001-3000 as our training, validation, and test sets, the data split would be essentially garbage. The way we solved this was to use numpy's built in `shuffle` function to shuffle our class label and feature's dictionary. This way, we can simply split our data by indices and take our desired 1-1000, 1001-2000, and 2001-3000 image sets. A quick series of print statements verifies a pretty decent split of 50-50 between class labels in our training, validation, and test sets. Finally, we standardized our data to a typical gaussian curve of mean 0 and standard deviation 1.

Next is the fun part; training our model. We want to prevent overfitting of our model, so we need to define some kind of regularization. We will use l2 regularization for our model. l2 regularization defines a penalty to the magnitude of the weights that it learns. This essentially forces the weights to be as minimal as possible. Another important parameter to consider is the C parameter. This term is the inverse of the regularization cost λ . This is the parameter that we are trying to optimize. To do this, we will train our model multiple times using different values for C, and we will use 5 fold cross validation to test for generalization error. We keep a running variable throughout our run that maintains the model with the best average accuracy over the 5 runs in 5 fold cross validation. The C value at the end that produces the best average accuracy in cross validation will be our final model, which we will then train again on our training data and finally test it with our test data set.

The best C value that was found through our iterative C optimization was 0.1 out of a tested range of values 0.001, 0.01, 0.1, 1, 10, 100, and 1000. This makes sense because there is a balance between the C

value being too high and/or too low. During the C optimization step we observed the average CV value for 0.001 at 0.9570 (the average accuracy) increase to 0.9630 for 0.01 C value, and then increase to 0.9650 for 0.1, where we then observed the average cross-validation score decrease to 0.9600 for a C value for 1, and then continue to decrease for all succeeding increasing C value scores. This makes sense, because a very small numerical C value will strongly regularize your model's weights, and result in an underfitting model that has a very low bias. A very large C value won't regularize much at all, and result in an overfit model that doesn't generalize very well to new data in the validation step. That is why we observed the highest average CV score for a C value that is in the middle of all of the C values that we tested.

The final step in our ML workflow is going to be to train a final logistic regression model using the best-found hyper-parameter C on a combined dataset containing the validation and training datasets, and to evaluate its performance on the test set. After training our model, we observed that, with the best found regularization hyper parameter of 0.1, we had a test error rate of 3.5 percent. This is a relatively reasonable result. We don't want our error rate to be too low (close to zero), because then our model would be a little too good and it would suggest problems in our code pertaining to issues such as data leakage. A 3.5 test error percentage is a good result for the simple task of binary classification on the MNIST dataset.

We were also asked to train a Random Forest model under binary classification using our data. Training a binary classification model in this case is going to be slightly different than logistic regression. In logistic regression, there were no hyper parameters relevant to the actual model itself; there were only hyper parameters for regularization and preventing overfitting. In the random forest model for scikit-learn, there are other hyper parameters. There are `n_estimators`, which is how many base learner trees we want to use. There is `max_depth`, which is how deep we want to grow our tree. And finally, there is `max_features`, which is how many of our original features we want to use to decide the split at each decision node. For the sake of simplicity, we will only optimize the `n_estimators` hyperparameter and set everything else to their default.

We cleaned our data in a manner identical to what we did for the logistic regression model. We then proceed to training our model. For training, we use 5 fold cross validation. This is essentially taking our training data, splitting it into 5 "folds", and over the course of 5 runs, we train our model on 4 of the folds and validate it on the 5th to test for overfitting and generalization error. We then do our actual "validation" on the validation set to report our model's ability to generalize. We store the results in lists, and then later plot the data using matplotlib.

We then do this for the entire range of base learners that we are testing. Finally, after we have our selected hyper parameter, we train a new random forest with the selected hyper parameter on the training + validation data. Finally, we evaluate it using the test data set.

The final step of model evaluation is comparing the test score to our training accuracy. We evaluated it using some common sense criteria. If the training accuracy is very high and the test accuracy is very low, then we consider our final model to have been overfitted. We define this to be if the training accuracy is greater than 99% and the test accuracy is less than 90%. If the test accuracy is only 5% lower than the training accuracy, then we consider the possibility of overfitting. If none of these are true, then that means our test accuracy is relatively close to our training accuracy, so we consider our model to be sufficient and we say that it generalizes well to new data.

Sources:

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

<https://www.ml-science.com/masking>

<https://stackoverflow.com/questions/47216388/efficiently-creating-masks-numpy-python>

<https://developers.google.com/machine-learning/crash-course/overfitting/regularization>